

fixtures/limit_class/ – T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/orchestration/limit_class.py` (plan.md T-B06; tasks.md T127/T135; research.md DEC-14; contracts/ agent-registry-and-handoff.md AR-005; FR-016). Task T127 (`tasks.md`) writes ONLY this RED test + these fixtures; the tool itself is T135, a later, separate task (Producer != Verifier, §11.4.240 — this file's author never implements `limit_class.py`).

A real contract DOES exist for this tool (unlike T109/tier_route.py): `specs/004-fast-dev-cycles/contracts/agent-registry-and-handoff.md` 's AR-005 clause + Components block fix the CLI verb (`limit_class.py --signal <raw> --out <class.json>`), the closed class set, and the exit codes. This fixture set borrows those field names/shapes from the real contract (T104/T108 pattern) rather than defining the entire interface from scratch (the T109/ tier_route.py pattern, which had no contract at all).

The classification rule this fixture set encodes (from research.md DEC-

005

+ contracts/agent-registry-and-handoff.md AR- + data-model.md's

limit_signal entity, restated precisely, never invented)

Closed class set (four members, DEC-14's own words): {rate-limited (retry-after present) | cap (spend/subscription/weekly cap: no retry-after, or a named weekly/subscription reset) | context-overflow (prompt too long) | other} .

1. **context-overflow** — the raw signal is an HTTP 400 **invalid_request** whose text names a token/prompt-length ceiling being exceeded ("prompt (is) too long ... N tokens > M maximum"). Structurally distinct from every other class: no HTTP 429, no **retry-after** , no cap/reset wording anywhere. Handling (AR-005 / plan T-B06): re-dispatch with the governance subset (DEC-12), **never** with the same prompt.
2. **cap** — an HTTP 429 that is EITHER missing a **retry-after** header OR carries a NAMED weekly/subscription reset instant (DEC-14's own disambiguator, stated as an OR of the two conditions). Handling: **never** retry on that alias; rebind the resume to an operational alias (§11.4.196 native-first order) and mark the alias cooled until its stated reset.
3. **rate-limited** — an HTTP 429 that carries a **retry-after** header AND no named weekly/subscription reset (the genuine transient/short-window case). Handling: backoff honouring **retry-after** , then resume on the same or another alias.
4. **other** — none of the above structurally matches; the classifier still emits a verdict (never refuses to classify) per the contract's exit code 1 note ("signal unparseable ... class **other** emitted with the raw signal").

Layout

Five self-contained fixtures, one JSON file each,

```
schema: "limit-class-fixture/v1" :
```

Fixture	Kind	What it proves
<code>lc_golden_cap_weekly.json</code>	golden	T127's own named golden scenario: the 2026-09-26 weekly-cap incident — a REAL row from <code>docs/requests/agent_registry.jsonl</code> — classifies as <code>cap</code> , never <code>rate-limited</code>
<code>lc_golden_context_overflow.json</code>	golden	T127's own named golden scenario: the haiku prompt-too-long incident — a REAL row from <code>docs/requests/agent_registry.jsonl</code> , corroborated by a second, independently-real occurrence captured 2026-09-30 by this project's own T114 dispatch — classifies as <code>context-overflow</code>
<code>lc_bad_cap_via_429_no_retry_after.json</code>	golden-bad	T127's own named golden-bad scenario ("a cap classified as rate-limited FAILs"): a second REAL row from the SAME 2026-09-26 weekly-cap incident, proving that a plausible mutation (ignore the <code>retry-after/reset</code> disambiguator, call every 429 <code>rate-limited</code>) misderives it

Fixture	Kind	What it proves
<code>lc_negc- trl_rate_limited_retry_after.json</code>	negat- ive- control	discriminator against the two <code>cap</code> fixtures above: an HTTP 429 that DOES carry a short <code>retry-after</code> and NO named reset classifies as <code>rate-limited</code> , proving the correct rule does not collapse every 429 into <code>cap</code> (the mirror-image failure mode of the golden-bad fixture). Honestly provenance: constructed — grep-verified (§11.4.115(G)): no real <code>retry-after</code> -bearing row exists anywhere in <code>docs/requests/agent_registry.jsonl</code> as of this dispatch.

Wire format (fixed by the real contract – `class.json` 's shape per

`data-model.md` 's `limit_signal` entity: `{class, raw, resets_at}`)

Each fixture's `expected_class` field is one of the four closed-set string literals above (hyphenated exactly as the contract spells them: `"rate-limited"`, `"cap"`, `"context-overflow"`, `"other"`). This suite's `derive_class` (embedded in `test_limit_class_red.sh`, never imported by nor shared with the not-yet-existing `orchestration/limit_class.py`, section 11.4.245) is the DERIVED oracle every fixture's own `expected_class` is independently re-verified

against before that value is trusted for anything downstream (mirroring

```
test_tier_routing_red.sh 'S derive_route /
```

```
test_governance_subset_red.sh 'S derive_selection precedent exactly).
```

The forward-compatible CLI this suite's "real-tool invocation" block exercises once T135 lands: `orchestration/limit_class.py --signal <raw> --out <class.json>` — the REAL, contract-fixed verb (not speculative, unlike T109/tier_route.py's provisional verb). Output shape `{class: str, raw: str, resets_at: str|"UNKNOWN"}` per data-model.md's `limit_signal` entity.

Provenance discipline (section
 . . . (G) / section . . .
 – CONSTRUCTED

reproductions never silently pass as OBSERVED ones)

Three of the five fixtures carry provenance: "observed" (their `raw_signal` is copied byte-for-byte from a real, cited row in `docs/requests/agent_registry.jsonl` or, for the context-overflow corroborating signal, from `specs/004-fast-dev-cycles/tasks.md` / `docs/CONTINUATION.md` / `docs/requests/history.md`). The negative-control fixture is honestly provenance: "constructed" — no real `retry-after` -bearing 429 row exists in this project's registry to date (confirmed by a `grep -i retry-after` control needle in the test script itself, not merely asserted here); the field shape it constructs (a `retry-after` seconds header on a 429) is the standard, well-established Anthropic API rate-limit convention this project's own constitution already cites (§11.4.187/§11.4.196(B) captured 429 signature; research.md DEC-14's own cited source, Anthropic rate limits, R5:97) — never fabricated at random.